

Introduction to Diffusion

Luxin Gabriel Matasaru

April 2025

What is diffusion?

What is diffusion?

A way of generating **stuff** that looks like training data.

What is diffusion?

We can generate images...



Figure: "image of a capybara dropping its papers while learning about diffusion"

What is diffusion?

...3D models...



Figure: try it out

What is diffusion?

... even animations!

"Sitting on the [sofa]"

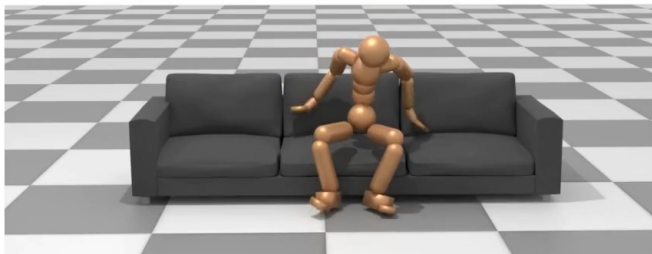


Figure: a paper from 2025

Another cool paper

What is diffusion?

arXiv:1503.03585v8 [cs.LG] 18 Nov 2015

Deep Unsupervised Learning using Nonequilibrium Thermodynamics

Yuichi Saito
Stanford University

Eric A. Weiss
University of California, Berkeley

Niru Mahowald
Stanford University

Surya Ganguli
Stanford University

YASCHA@STANFORD.EDU

LAWEISS@BERKELEY.EDU

NIRUM@STANFORD.EDU

SGANGULI@STANFORD.EDU

Abstract

A central problem in machine learning involves modeling complex data sets using highly flexible families of probability distributions in which learning, sampling, inference, and evaluation are still analytically or computationally intractable. Here, we develop an approach that simultaneously achieves both flexibility and tractability. The essential idea, inspired by non-equilibrium statistical physics, is to systematically and slowly destroy structure in a data distribution through an iterative forward diffusion process. We then learn a reverse diffusion process that restores structure in data, yielding a highly flexible and tractable generative model of the data. This approach allows us to rapidly learn, sample from, and evaluate probabilities in deep generative models with thousands of layers or time steps, as well as to compute conditional and posterior probabilities under the learned model. We additionally release an open source reference implementation of the algorithm.

1. Introduction

Historically, probabilistic models suffer from a tradeoff between two conflicting objectives: tractability and flexibility. Models that are tractable can be analytically evaluated and easily fit to data (e.g. a Gaussian or Laplace). However,

Proceedings of the 32nd International Conference on Machine Learning, Lille, France, 2015. JMLR: W&CP volume 37. Copyright 2015 by the author(s).

these models are unable to apply describe structure in rich datasets. On the other hand, models that are flexible can be modeled to fit structure in arbitrary data. For example, we can define models in terms of any (non-negative) function $q(x)$ yielding the flexible distribution $p(x) = \frac{q(x)}{Z}$, where Z is a normalization constant. However, computing this normalization constant is generally intractable. Evaluating, training, or drawing samples from such flexible models typically requires a very expensive Monte Carlo process.

A variety of analytic approximations exist which ameliorate, but do not remove, this tradeoff—for instance mean field theory and its expansions (T, 1982; Tanaka, 1998), variational Bayes (Jordan et al., 1999), contrastive divergence (Welling & Hinton, 2002; Hinton, 2002), minimum probability flow (Sohl-Dickstein et al., 2011b), minimum KL contraction (Lyu, 2011), proper scoring rules (Gershman & Raftery, 2007; Puri et al., 2012), score matching (Hyvärinen, 2005), pseudolikelihood (Besag, 1975), loopy belief propagation (Murphy et al., 1999), and many, many more. Non-parametric methods (Gershman & Blei, 2012) can also be very effective.

1.1. Diffusion probabilistic models

We present a novel way to define probabilistic models that allows:

1. extreme flexibility in model structure,

2. exact sampling.

Non-parametric methods can be seen as trading off smoothness between intractable and flexible models. For instance, a non-parametric Gaussian mixture model will represent a small amount of data using a single Gaussian, but may represent infinite data as a mixture of an infinite number of Gaussians.

(a) link

arXiv:2006.11239v2 [cs.LG] 16 Dec 2020

Denoising Diffusion Probabilistic Models

Jonathan Ho
UC Berkeley
jonathanho@berkeley.edu

Ajoy Jain
UC Berkeley
ajayj@berkeley.edu

Pieter Abbeel
UC Berkeley
pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decomposition scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at <https://github.com/hojonathanho/diffusion>.

1 Introduction

Deep generative models of all kinds have recently exhibited high quality samples in a wide variety of data modalities. Generative adversarial networks (GANs), autoregressive models, flows, and variational autoencoders (VAEs) have synthesized striking image and audio samples (14, 27, 3, 58, 38, 25, 10, 32, 44, 57, 26, 33, 45), and there have been remarkable advances in energy-based modeling and score matching that have produced images comparable to those of GANs (11, 55).

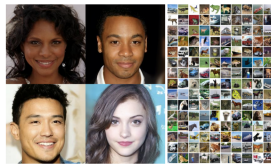


Figure 1: Generated samples on CelebA-HQ 256 x 256 (left) and unconditional CIFAR10 (right)

34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.

(b) link

Table of contents

- ▶ Intuition
- ▶ Mathematical proof
- ▶ Optimizations

Intuition

- ▶ Let's say we want to generate images of cats.
- ▶ Assume we have a function which can tell us, for any image, the **likelihood** that it's a cat.

Intuition

- ▶ Let's say we want to generate images of cats.
- ▶ Assume we have a function which can tell us, for any image, the **likelihood** that it's a cat.



(e) $f() = 0.9$



(f) $f() = 0.3$

Intuition

- ▶ Let's say we want to generate images of cats.
- ▶ Assume we have a function which can tell us, for any image, the **likelihood** that it's a cat.



(g) $f() = 0.9$



(h) $f() = 0.3$

- ▶ We can use f to generate images of cats.

Intuition

- ▶ We are actually interested in $\nabla f_x(x_0)$, the gradient of f w.r.t. the pixels of our image.

Intuition

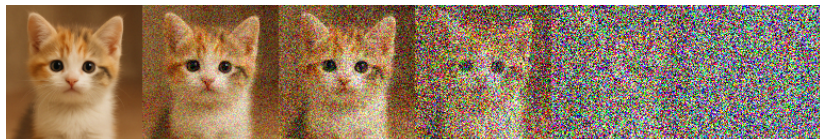
- ▶ We are actually interested in $\nabla f_x(x_0)$, the gradient of f w.r.t. the pixels of our image.
- ▶ For now f is just a black box, so we can't compute the gradient...

Intuition

- ▶ We are actually interested in $\nabla f_x(x_0)$, the gradient of f w.r.t. the pixels of our image.
- ▶ For now f is just a black box, so we can't compute the gradient...
- ▶ ... but we can approximate it by a neural network.

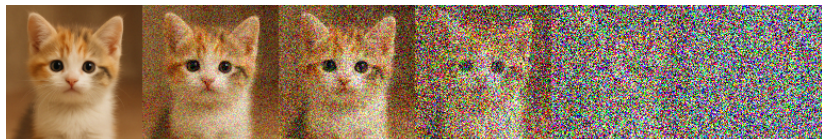
Intuition

- ▶ Generate images that "vaguely resemble a cat" by adding noise ($\mathcal{N}(0, \sigma^2)$) to a good image.



Intuition

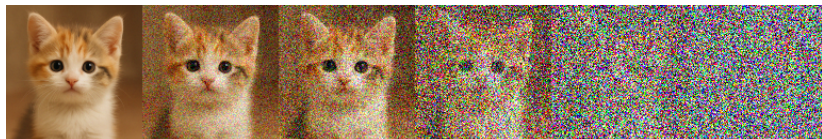
- ▶ Generate images that "vaguely resemble a cat" by adding noise ($\mathcal{N}(0, \sigma^2)$) to a good image.



- ▶ Model should output the change needed for each pixel such that we get an image of a cat (i.e. the noise added!).

Intuition

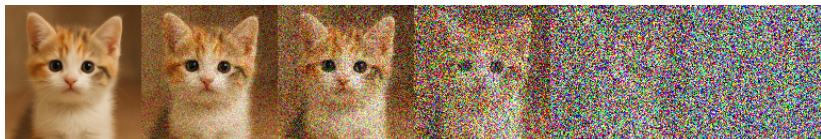
- ▶ Generate images that "vaguely resemble a cat" by adding noise ($\mathcal{N}(0, \sigma^2)$) to a good image.



- ▶ Model should output the change needed for each pixel such that we get an image of a cat (i.e. the noise added!).
- ▶ Define the loss function as the MSE between the predicted noise and the actual noise.

Intuition

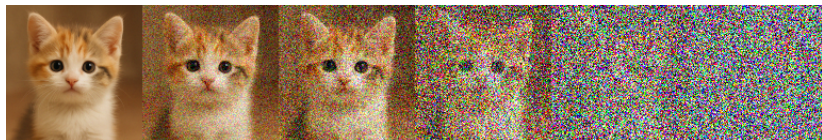
- ▶ Generate images that "vaguely resemble a cat" by adding noise ($\mathcal{N}(0, \sigma^2)$) to a good image.



- ▶ Model should output the change needed for each pixel such that we get an image of a cat (i.e. the noise added!).
- ▶ Define the loss function as the MSE between the predicted noise and the actual noise.
- ▶ Train model to minimize loss function.

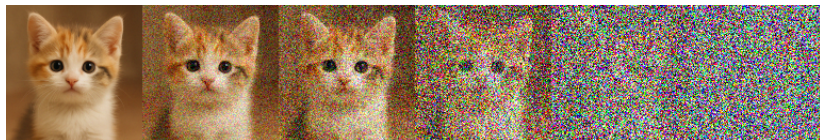
Intuition

- ▶ Problem: there isn't enough information to reliably go from a very noisy image to a clean one.



Intuition

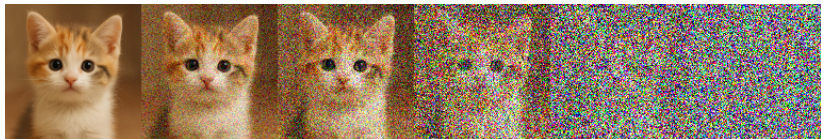
- ▶ Problem: there isn't enough information to reliably go from a very noisy image to a clean one.



- ▶ Then only try to predict some of the noise (say until the next "previous" step).

Intuition

- ▶ Problem: there isn't enough information to reliably go from a very noisy image to a clean one.



- ▶ Then only try to predict some of the noise (say until the next "previous" step).
- ▶ We could even tell the model how much noise was added ($\sigma^2 = \beta_t$).

Intuition

- ▶ Finally, give a $\mathcal{N}(0, 1)$ image to the trained model.

Intuition

- ▶ Finally, give a $\mathcal{N}(0, 1)$ image to the trained model.
- ▶ It should denoise it gradually, and produce something resembling a cat.

Intuition

- ▶ Finally, give a $\mathcal{N}(0, 1)$ image to the trained model.
- ▶ It should denoise it gradually, and produce something resembling a cat.

****In theory****

Intuition

- ▶ Adding the noise from timestep $t - 1$ to t is called the **forward process**.
- ▶ The amount of noise we add at each timestep, $\sigma^2 = \beta_t$, forms $(\beta_t)_{1 \leq t \leq T}$, the **noise/variance schedule**.
- ▶ Gradually denoising the image at timestep t to get the image at $t - 1$ is called the **reverse process**.

Intuition

- ▶ Adding the noise from timestep $t - 1$ to t is called the **forward process**.
- ▶ The amount of noise we add at each timestep, $\sigma^2 = \beta_t$, forms $(\beta_t)_{1 \leq t \leq T}$, the **noise/variance schedule**.
- ▶ Gradually denoising the image at timestep t to get the image at $t - 1$ is called the **reverse process**.

Remember the analogy with dye diffusing in a glass of water.

Mathematical formalization

- ▶ Let $q(x_0)$ be the probability distribution of our dataset. (This is what we wish to approximate with our model)

$$x_0 \sim q(x_0)$$

Mathematical formalization

- ▶ Let $q(x_0)$ be the probability distribution of our dataset. (This is what we wish to approximate with our model)
- ▶ We will generally denote by x_0 a random variable with realizations in the set of images from the dataset.

$$x_0 \sim q(x_0)$$

Mathematical formalization - forward process

- ▶ Given a sequence $x_0, x_1, \dots, x_T$ where $x_t \sim q(x_t|x_{t-1}) \forall 1 \leq t \leq T$.

Mathematical formalization - forward process

- ▶ Given a sequence $x_0, x_1, \dots, x_T$ where $x_t \sim q(x_t|x_{t-1}) \forall 1 \leq t \leq T$.
- ▶ We define $q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$.

Mathematical formalization - forward process

- ▶ Given a sequence $x_0, x_1, \dots, x_T$ where $x_t \sim q(x_t|x_{t-1}) \forall 1 \leq t \leq T$.
- ▶ We define $q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$.
- ▶ Then, forward process is

$$q(x_{1:T}|x_0) := \frac{q(x_0, x_1, x_2, \dots, x_T)}{q(x_0)} = \prod_{t=1}^T q(x_t|x_{t-1})$$

Mathematical formalization - forward process

- ▶ Given a sequence $x_0, x_1, \dots, x_T$ where $x_t \sim q(x_t|x_{t-1}) \forall 1 \leq t \leq T$.
- ▶ We define $q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$.
- ▶ Then, forward process is

$$q(x_{1:T}|x_0) := \frac{q(x_0, x_1, x_2, \dots, x_T)}{q(x_0)} = \prod_{t=1}^T q(x_t|x_{t-1})$$

- ▶ By defining $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ we can prove by induction that

$$q(x_t|x_0) := \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

Mathematical formalization - forward process

- ▶ Given a sequence $x_0, x_1, \dots, x_T$ where $x_t \sim q(x_t|x_{t-1}) \forall 1 \leq t \leq T$.
- ▶ We define $q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$.
- ▶ Then, forward process is

$$q(x_{1:T}|x_0) := \frac{q(x_0, x_1, x_2, \dots, x_T)}{q(x_0)} = \prod_{t=1}^T q(x_t|x_{t-1})$$

- ▶ By defining $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ we can prove by induction that

$$q(x_t|x_0) := \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$$

In mathspeak, $x_0, x_1, \dots, x_T$ form a **Markov chain** with normal transitions $q(x_t|x_{t-1})$. Moreover, we have:

$$q(x_0, x_1, x_2, \dots, x_T) = q(x_0) \prod_{t=1}^T q(x_t|x_{t-1})$$

Mathematical formalization - reverse process

- ▶ Same as before, we define the reverse process as a Markov chain.

Mathematical formalization - reverse process

- ▶ Same as before, we define the reverse process as a Markov chain.
- ▶ Start from pure noise: $x_T \sim p(x_T) := \mathcal{N}(0, I)$

Mathematical formalization - reverse process

- ▶ Same as before, we define the reverse process as a Markov chain.
- ▶ Start from pure noise: $x_T \sim p(x_T) := \mathcal{N}(0, I)$
- ▶ And gradually recover

$$x_{t-1} \sim p_{\theta}(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

Mathematical formalization - reverse process

- ▶ Same as before, we define the reverse process as a Markov chain.
- ▶ Start from pure noise: $x_T \sim p(x_T) := \mathcal{N}(0, I)$
- ▶ And gradually recover

$$x_{t-1} \sim p_{\theta}(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

- ▶ We would like to maximize $\log p_{\theta}(x_0) = \log \int p_{\theta}(x_{0:T}) dx_{1:T}$.

Values with subscript θ are learned by the model. (θ represents the weights of our neural network)

Mathematical formalization - reverse process

- ▶ Same as before, we define the reverse process as a Markov chain.
- ▶ Start from pure noise: $x_T \sim p(x_T) := \mathcal{N}(0, I)$
- ▶ And gradually recover

$$x_{t-1} \sim p_\theta(x_{t-1}|x_t) := \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

- ▶ We would like to maximize $\log p_\theta(x_0) = \log \int p_\theta(x_{0:T}) dx_{1:T}$.
- ▶ However, this is not easily computable :(

Values with subscript θ are learned by the model. (θ represents the weights of our neural network)

Mathematical formalization - loss function

- ▶ We can however compute a lower bound and then maximize that!

Mathematical formalization - loss function

- We can however compute a lower bound and then maximize that!

$$\begin{aligned}\log p_{\theta}(x_0) &= \log \int p_{\theta}(x_{0:T}) dx_{1:T} \\ &= \log \int p_{\theta}(x_{0:T}) \frac{q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)} dx_{1:T} \\ &= \log \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right] \\ &\geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right]\end{aligned}$$

Mathematical formalization - loss function

- We can however compute a lower bound and then maximize that!

$$\begin{aligned}\log p_{\theta}(x_0) &= \log \int p_{\theta}(x_{0:T}) dx_{1:T} \\ &= \log \int p_{\theta}(x_{0:T}) \frac{q(x_{1:T}|x_0)}{q(x_{1:T}|x_0)} dx_{1:T} \\ &= \log \mathbb{E}_{q(x_{1:T}|x_0)} \left[\frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right] \\ &\geq \mathbb{E}_{q(x_{1:T}|x_0)} \left[\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right]\end{aligned}$$

We actually prefer to minimize the negative expected log likelihood:

$$\mathbb{E}_{q(x_0)} [-\log p_{\theta}(x_0)] \leq \mathbb{E}_{q(x_0)} \left[\mathbb{E}_{q(x_{1:T}|x_0)} \left[-\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right] \right]$$

Mathematical formalization - loss function

Let's rewrite this in a simpler form:

Mathematical formalization - loss function

Let's rewrite this in a simpler form:

$$\begin{aligned}-\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} &= -\log \frac{p_{\theta}(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t)}{\prod_{t=1}^T q(x_t|x_{t-1})} \\&= -\log p_{\theta}(x_T) + \sum_{t=1}^T \frac{q(x_t|x_{t-1})}{p_{\theta}(x_{t-1}|x_t)} \\&= -\log p_{\theta}(x_T) + \sum_{t=2}^T \frac{q(x_t|x_{t-1})}{p_{\theta}(x_{t-1}|x_t)} + \frac{q(x_1|x_0)}{p_{\theta}(x_0|x_1)}\end{aligned}$$

Mathematical formalization - loss function

Let's rewrite this in a simpler form:

$$\begin{aligned}-\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} &= -\log \frac{p_{\theta}(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t)}{\prod_{t=1}^T q(x_t|x_{t-1})} \\&= -\log p_{\theta}(x_T) + \sum_{t=1}^T \frac{q(x_t|x_{t-1})}{p_{\theta}(x_{t-1}|x_t)} \\&= -\log p_{\theta}(x_T) + \sum_{t=2}^T \frac{q(x_t|x_{t-1})}{p_{\theta}(x_{t-1}|x_t)} + \frac{q(x_1|x_0)}{p_{\theta}(x_0|x_1)}\end{aligned}$$

Now, using Bayes rule:

$$\begin{aligned}q(x_t|x_{t-1}) &= \frac{q(x_{t-1}|x_t)q(x_t)}{q(x_{t-1})} \\&= \frac{q(x_{t-1}|x_t, x_0)q(x_t|x_0)}{q(x_{t-1}|x_0)}\end{aligned}$$

Mathematical formalization - loss function

Let's rewrite this in a simpler form:

$$-\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} = -\log p_{\theta}(x_T) + \sum_{t=2}^T \frac{q(x_t|x_{t-1})}{p_{\theta}(x_{t-1}|x_t)} + \frac{q(x_1|x_0)}{p_{\theta}(x_0|x_1)}$$

Mathematical formalization - loss function

Let's rewrite this in a simpler form:

$$\begin{aligned} &= -\log p_{\theta}(x_T) + \sum_{t=2}^T \log \frac{q(x_t|x_{t-1})}{p_{\theta}(x_{t-1}|x_t)} + \log \frac{q(x_1|x_0)}{p_{\theta}(x_0|x_1)} \\ &= -\log p_{\theta}(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)q(x_t|x_0)}{p_{\theta}(x_{t-1}|x_t)q(x_{t-1}|x_0)} + \log \frac{q(x_1|x_0)}{p_{\theta}(x_0|x_1)} \\ &= -\log p_{\theta}(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)} + \sum_{t=2}^T \log \frac{q(x_t|x_0)}{q(x_{t-1}|x_0)} + \log \frac{q(x_1|x_0)}{p_{\theta}(x_0|x_1)} \\ &= -\log p_{\theta}(x_T) + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)} + \log \frac{q(x_T|x_0)}{q(x_1|x_0)} + \log \frac{q(x_1|x_0)}{p_{\theta}(x_0|x_1)} \\ &= \log \frac{q(x_T|x_0)}{p_{\theta}(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)} - \log p_{\theta}(x_0|x_1) \end{aligned}$$

Mathematical formalization - loss function

Finally, we ended up with:

$$\mathbb{E}_{q(x_0)} [-\log p_\theta(x_0)] \leq \\ \mathbb{E}_{q(x_0:T)} \left[\log \frac{q(x_T|x_0)}{p_\theta(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} - \log p_\theta(x_0|x_1) \right]$$

Mathematical formalization - loss function

Finally, we ended up with:

$$\mathbb{E}_{q(x_0)} [-\log p_\theta(x_0)] \leq \mathbb{E}_{q(x_0:T)} \left[\log \frac{q(x_T|x_0)}{p_\theta(x_T)} + \sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_\theta(x_{t-1}|x_t)} - \log p_\theta(x_0|x_1) \right]$$

Now each term is actually computable!

Mathematical formalization - loss function

We will focus on

$$\sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)}$$

Mathematical formalization - loss function

We will focus on

$$\sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)}$$

► We already know

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \beta I)$$

Mathematical formalization - loss function

We will focus on

$$\sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)}$$

- We already know

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \beta I)$$

- And $q(x_{t-1}|x_t, x_0)$ has a closed form solution:

$$q(x_{t-1}|x_t, x_0) = \mathcal{N}(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

$$\begin{aligned}\tilde{\mu}_t(x_t, x_0) &= \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_{t-1}}x_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}x_t \\ &= \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\epsilon \right)\end{aligned}$$

Mathematical formalization - loss function

- We want to minimize

$$\sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)}$$

Mathematical formalization - loss function

- ▶ We want to minimize

$$\sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)}$$

- ▶ So $q(x_{t-1}|x_t, x_0)$ and $p_{\theta}(x_{t-1}|x_t)$ need to be as close as possible.

Mathematical formalization - loss function

- ▶ We want to minimize

$$\sum_{t=2}^T \log \frac{q(x_{t-1}|x_t, x_0)}{p_{\theta}(x_{t-1}|x_t)}$$

- ▶ So $q(x_{t-1}|x_t, x_0)$ and $p_{\theta}(x_{t-1}|x_t)$ need to be as close as possible.
- ▶ We can measure this using $MSE(\tilde{\mu}_t(x_t, x_0), \mu_{\theta}(x_t, t))$

Mathematical formalization - loss function

$$\begin{aligned}\mathcal{L} &= \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(x_t, x_0), \mu_\theta(x_t, t)\|^2 \\ &= \left\| \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right) + \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) \right\|^2 \\ &\propto \|\epsilon - \epsilon_\theta(x_t, t)\|^2\end{aligned}$$

Mathematical formalization - conclusions

- ▶ We actually proved that predicting the next image (the mean) is equivalent to predicting the noise.
- ▶ It turns out that predicting the noise gives much better results

Optimizations

- ▶ Train the model using text embeddings as well.
- ▶ Encode the image into latent space and run diffusion on that.
- ▶ Learn to also predict the variance during the reverse process.
- ▶ Optimize the noise schedule.
- ▶ Guidance.